CV Key Concepts

An image is represented in computers as a matrix (grid) of numbers, where each number corresponds to the intensity or color value of a pixel. In grayscale, it's a 2D matrix of values from 0–255, while in color images it's a 3D matrix with separate channels (Red, Green, Blue or BGR in OpenCV).

Image Representation Basics

- **Grayscale Image** → 2D matrix
  - Each pixel is a single number (0 = black, 255 = white).
  - Example: a 3×3 grayscale image might look like:

[

0	128	255
64	192	128
255	255	0

]

- **Color Image (RGB/BGR)** → 3D matrix
  - Each pixel has 3 values (R, G, B channels).
  - Example: Pixel `[255, 0, 0]` = pure red.
- **Coordinate System**
  - Origin `(0,0)` is at the **top-left corner**.
  - `x` increases to the right, `y` increases downward.
  - Access pixel: `image[y, x]`.

In Python (OpenCV + NumPy)

```
import cv2
import numpy as np

# Load grayscale image
img_gray = cv2.imread('sample.jpg', 0)
print(img_gray.shape)   # (rows, cols)
print(img_gray[0,0])    # intensity of top-left pixel

# Load color image
img_color = cv2.imread('sample.jpg')
print(img_color.shape)  # (rows, cols, 3)
print(img_color[0,0])   # BGR values of top-left pixel
```

- **Grayscale** → 2D NumPy array.
- **Color** → 3D NumPy array with 3 channels.
- OpenCV uses **BGR order** (Blue, Green, Red), while Matplotlib expects RGB.

HSV

HSV stands for Hue, Saturation, and Value. It's a way of representing colors that's closer to how humans perceive them compared to RGB.

- **Hue (H):**
  - The *type of color* (red, green, blue, etc.).
  - Measured as an angle on a color wheel (0° = red, 120° = green, 240° = blue).
- **Saturation (S):**
  - How *intense or pure* the color is.
  - 0% = completely gray, 100% = fully vivid color.
- **Value (V):**
  - The *brightness* of the color.
  - 0% = black, 100% = brightest version of the hue.

Why HSV is Useful

- **RGB** mixes colors using red, green, and blue light values — great for screens, but not intuitive for humans.
- **HSV** separates color into intuitive components:
  - Hue → which color family.
  - Saturation → how colorful it is.
  - Value → how bright it is.

This makes HSV especially useful in **computer vision tasks** like color detection, segmentation, and image editing. For example, detecting ripe fruit is easier in HSV because you can isolate the hue range for "yellow" or "red" without worrying about brightness changes.

Contours are curves joining continuous points along the boundary of objects with the same intensity or color. Contour analysis in image processing refers to extracting and studying the boundaries of objects in an image. It is widely used for shape recognition, object detection, and classification, with features like area, perimeter, centroid, bounding boxes, and convex hulls being key descriptors.

What is a Kernel?

In image processing, a kernel is a small matrix (like a 3×3 or 5×5 grid of numbers) that slides over an image to modify it — for example, to blur, sharpen, or detect edges. Each kernel defines how a pixel and its neighbors are combined to produce a new output pixel.

- Definition: A kernel (also called a filter, mask, or convolution matrix) is a set of numbers arranged in a small grid.
- Purpose: It determines how pixel values in an image are weighted and combined during convolution.
- Operation:
    - Place the kernel over a pixel and its neighbors.
    - Multiply each kernel value by the corresponding pixel value.
    - Sum the results → this becomes the new pixel value.
- Size: Common kernel sizes are 3×3, 5×5, or 7×7. Larger kernels capture more context but are slower to compute.

Examples of Kernels

Kernel Type	Matrix Example	Effect on Image
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	Leaves image unchanged
Blur (Box Filter)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	Smooths image, reduces detail

Kernel Type	Matrix Example	Effect on Image
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Enhances edges and fine details
Edge Detection (Sobel X)	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	Highlights vertical edges
Edge Detection (Sobel Y)	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	Highlights horizontal edges

Why Kernels Matter

- Feature Extraction: Kernels help detect edges, corners, and textures.
- Image Enhancement: Used for sharpening, smoothing, or embossing.
- Deep Learning: In CNNs, kernels are learned automatically to capture patterns like shapes or objects.

Summary

- A kernel is a recipe for how to combine pixel values.
- Different kernels → different effects (blur, sharpen, edge detection).
- Kernels are the foundation of both classical image processing and modern deep learning (CNNs).

Convolution (Image Filtering)

Convolution is a way of scanning an image with a small filter (kernel) to highlight or change features like edges, blur, or sharpen. Histograms show the distribution of pixel intensities in an image, while thresholding is a technique to convert an image into black-and-white (binary) by setting a cutoff value.

- Definition: Convolution is a mathematical operation where a small matrix (kernel) slides over the image, multiplying and summing pixel values to produce a new output.
- Purpose: Extracts features such as edges, textures, or smoothness.
- Different kernels do different jobs:
    - Blur kernel → smooths the image
    - Sharpen kernel → makes edges clearer
    - Edge detection kernel → finds boundaries
- Formula (2D convolution):

$$(I*K)(x,y) = \sum_m \sum_n I(x,-my-n)\cdot K(m,n)$$

where $I$is the image and $K$is the kernel.

Histograms

- Definition: A histogram is a graph showing how many pixels in an image have each intensity value.
- Grayscale histogram: X-axis = intensity (0–255), Y-axis = count of pixels.
- Uses:
    - Detect brightness and contrast levels.
    - Identify whether an image is underexposed (too dark) or overexposed (too bright).
    - Basis for enhancement techniques like histogram equalization.

Thresholding

- Definition: Thresholding converts a grayscale image into a binary image (black and white) by setting a cutoff value.
- Process:
    - If pixel intensity > threshold → set to white (255).
    - If pixel intensity ≤ threshold → set to black (0).
- Types:
    - Global thresholding: One fixed cutoff for the entire image.
    - Adaptive thresholding: Different cutoffs for different regions, useful when lighting varies.
- Applications:
    - Document scanning (separating text from background).
    - Medical imaging (highlighting tumors).
    - Object detection (separating foreground from background).

Summary

- Convolution → applies filters to extract features.
- Histograms → visualize pixel intensity distribution.
- Thresholding → simplifies images into binary form for easier analysis.

Morphological Operations

Morphological operations are image processing techniques that focus on the shape and structure of objects in an image. They are especially useful when working with binary images (black-and-white) but can also be applied to grayscale.

Morphological operations use a structuring element (a small shape like a square, circle, or cross) to probe the image. By sliding this structuring element across the image, we can modify the shapes of objects — either expanding, shrinking, or cleaning them.

Common Morphological Operations

1. Dilation
    - Expands white regions (foreground).
    - Useful for connecting broken parts of objects.
    - Example: Filling gaps in text characters.
2. Erosion
    - Shrinks white regions.
    - Useful for removing small noise.
    - Example: Thinning lines or removing stray dots.
3. Opening
    - Erosion followed by dilation.
    - Removes small noise while keeping main shapes intact.
    - Example: Cleaning up scanned documents.
4. Closing
    - Dilation followed by erosion.
    - Fills small holes inside objects.
    - Example: Filling gaps in letters or shapes.

Applications

- Document processing: Remove noise, enhance text readability.

- Medical imaging: Highlight structures like cells or tissues.
- Industrial inspection: Detect defects in materials.
- Object detection: Clean up shapes before contour analysis.

Think of morphological operations like editing a drawing with a marker and eraser:

- Dilation = making strokes thicker.
- Erosion = making strokes thinner.
- Opening = erasing tiny specks.
- Closing = filling tiny gaps.

Object Tracking

- Definition: Continuously locating objects across video frames.
- Approaches:
    - Detectionbased tracking: Detect objects in each frame (e.g., YOLO, Faster RCNN) and link detections.
    - Trackingbymotion: Use optical flow or correlation filters to follow objects.
    - Hybrid methods: Combine detection + prediction (e.g., SORT, DeepSORT).
- Challenges: Occlusion, scale changes, multiple objects, cluttered backgrounds.
- Applications: Surveillance, autonomous driving, sports analytics.

Edge Detection

- Purpose: Identify boundaries in images where intensity changes sharply.
- Methods: Sobel, Canny, Laplacian filters.
- Applications: Object segmentation, lane detection, medical imaging.
- Note: Edges are distinctive but not always unique landmarks (sliding along an edge looks similar).

Corner Detection

- Purpose: Find points where edges meet, i.e., regions with intensity variation in multiple directions.
- Methods: Harris Corner Detector, FAST, ShiTomasi.
- Applications: Image matching, tracking, panorama stitching.
- Why useful: Corners are repeatable and uniquely localizable compared to flat patches or edges.

SIFT (ScaleInvariant Feature Transform)

- Purpose: Detect and describe key points robust to scale and rotation changes.
- Pipeline:
  - Build Gaussian scale space.
  - Detect extrema (Difference of Gaussians).
  - Assign orientation.
  - Create 128dimensional descriptor vector.
- Applications: Object recognition, image stitching, 3D reconstruction.
- Strength: Robust to scale/rotation; Limit: computationally heavy.

SURF (SpeededUp Robust Features) is a computer vision algorithm used for detecting and describing local features in images. It was introduced in 2006 as a faster alternative to SIFT, offering scale and rotation invariance while being computationally efficient.

- Feature Detection: Finds interest points (corners, blobs, junctions) using a Hessian matrixbased measure.
- Feature Description: Represents the neighbourhood around each key point as a compact vector (64 or 128 dimensions).
- Matching: Compares descriptors across images to establish correspondences.

HoG (Histogram of Oriented Gradients)

- Purpose: Capture edge orientation distributions in local regions.
- Pipeline:
  - Compute gradients.
  - Divide image into cells (e.g., 8×8).
  - Build orientation histograms.
  - Normalize over blocks.
- Applications: Pedestrian detection, object classification.
- Strength: Lightweight and interpretable; Limit: sensitive to scale changes.

Stereo Vision

- Purpose: Use two images from slightly different viewpoints to estimate depth.
- Concept: Match corresponding points between left and right images.
- Applications: 3D reconstruction, autonomous driving, robotics.
- Challenge: Finding reliable correspondences in texture less regions.

Epipolar Geometry

- Purpose: Mathematical framework describing the relationship between two camera views.
- Key Idea: For a point in one image, its corresponding point in the other must lie on the epipolar line.
- Tools: Essential matrix (calibrated cameras), Fundamental matrix (uncalibrated).
- Applications: Stereo matching, structurefrommotion, camera calibration.

Optical Flow

- Definition: Estimates the apparent motion of pixels between consecutive frames.
- How it works: Assumes brightness of a pixel remains constant as it moves; calculates displacement vectors.
- Methods:
    - *Lucas–Kanade*: Local neighbourhood approximation.
    - *Horn–Schunck*: Global smoothness constraint.
- Applications: Gesture recognition, motion detection, video stabilization.
- Limitations: Sensitive to lighting changes and occlusions.

Kalman Filters

- Definition: A recursive algorithm that predicts and corrects the state of a moving object over time.
- How it works:
    - Prediction step: Estimates next position based on motion model.
    - Update step: Corrects prediction using observed measurements.
- Strengths: Handles noisy sensor data, smooths trajectories.
- Applications: Tracking vehicles, drones, or pedestrians in real time.
- Note: Works best for linear motion with Gaussian noise; extended Kalman filters handle nonlinear cases.